







What does this get us?

when (src) {  **src.balance += 10; }**

when (dst) {  **dst.balance += 10; }**

4

6

when(dst) {  **print(dst.balance) }**

when(src) {  **print(src.balance) }**

src=0, dst=0



src=0, dst=10



src=10, dst=0



src=10, dst=10



Order

Permitted



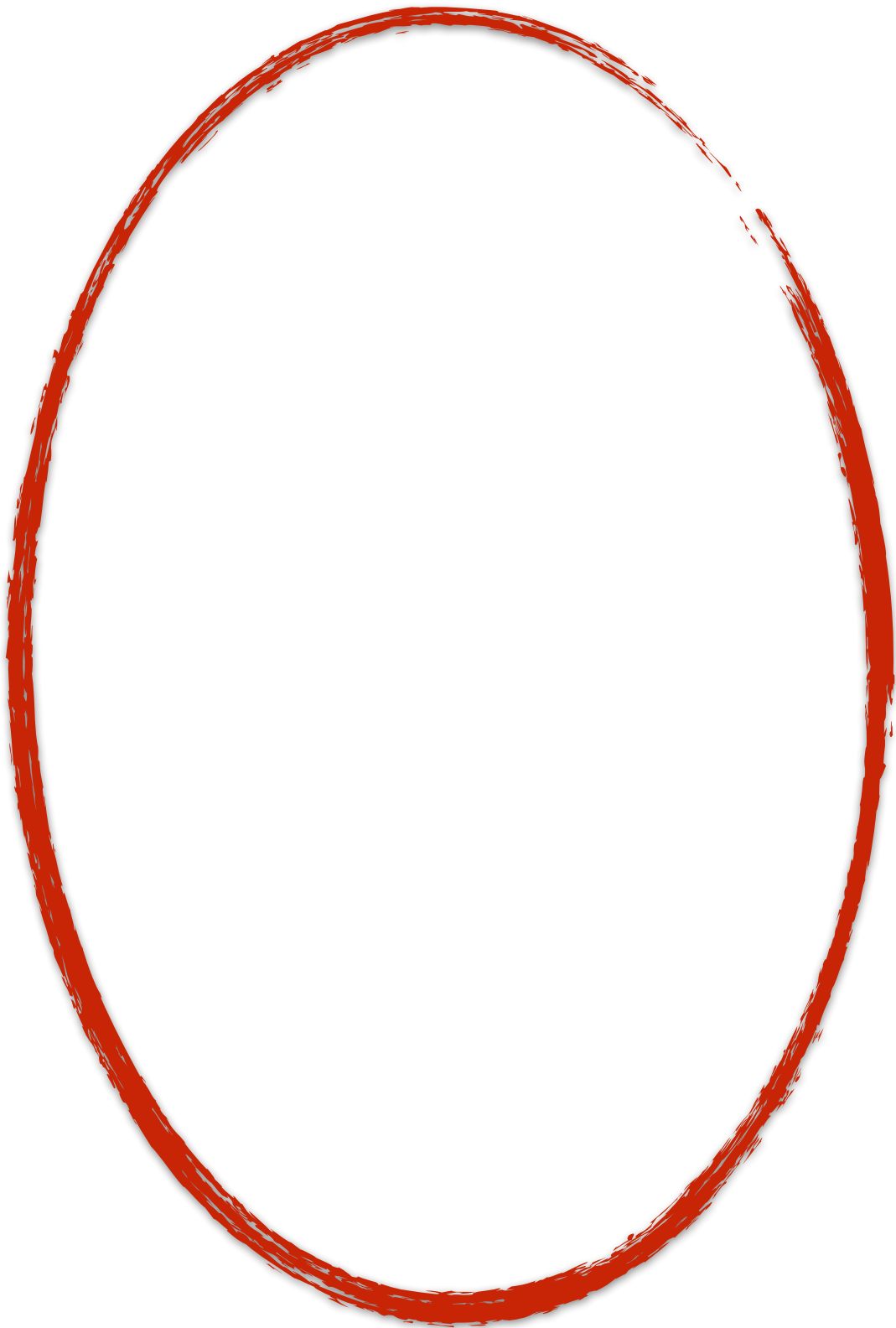






updatepath(src then dst)

Diagnostic path (dst then src)





What does this get us?

Update path (src then dst)

```
when(src) { A src.balance += 10; }  
when(dst) { B dst.balance += 10; }
```

Diagnostic path (dst then src)

```
when(dst) { C print(dst.balance) }  
when(src) { D print(src.balance) }
```

	src=0, dst=0	src=0, dst=10	src=10, dst=0	src=10, dst=10
Order	C C C D D D	B B B D D D	A A A C C C	A A A B B B
	B D D A A A	C D D A A B	C C D C A B	B B D C A C
	D C B A C A	C A C A B C	B D C B D B	C D B C D C
	A A A B C B	A C A C B A	D B B D C A	D C C C D A
	A B B C D C	A C A C B A	D B B D C A	D C C C D A
	A B B C D C	A C A C B A	D B B D C A	D C C C D A
Permitted	✓	✓	✓	✓

What does this get us?

We can reason about program transformations

```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```

Can we release cows early?

```
when(src) { E
  when(dst) { F }
}
when(dst) { G }
```

```
when(dst) { G }
when(src) { E
  when(dst) { F }
}
```

Can we reorder behaviours?